

短信验证码登录功能完整指南—前端

1. 项目概览

项目名称： my-vue-app
技术栈： Vue 3 + TypeScript + Vite + Element Plus

2. 初始化 Vue 项目

2.1. 执行命令

```
1 npm create vue@latest my-vue-app
```

解释：

- 1. **npm create** : 这是 npm 7+ 版本提供的新命令，相当于 **npx create-vue@latest**
- 2. **vue@latest** : 使用最新版本的 Vue 官方脚手架工具
- 3. **my-vue-app** : 项目目录名称

2.2. 脚手架选择

在执行命令后，Vue CLI 会询问一系列配置选项，暂时选择 TS 和 Router 即可。



3. 进入项目目录并安装依赖

3.1. 执行命令

```
1 cd my-vue-app
2 npm install
```

3.2. 依赖分析

3.2.1. 核心依赖 (dependencies)

注意：实际代码中，json 文件不可以加注释，这里只是为了解释方便

```
1 {
2   "vue": "^3.5.21", // Vue 3 核心框架
3   "vue-router": "^4.5.1", // 官方路由管理器
4   "axios": "^1.12.2", // HTTP 客户端库
5   "element-plus": "^2.11.3", // UI 组件库
6   "@element-plus/icons-vue": "^2.3.2" // Element Plus 图标库
7 }
```

3.2.2. 开发依赖 (devDependencies)

```
1 {
2   "@vitejs/plugin-vue": "^6.0.1", // Vite 的 Vue 插件
3   "@vue/tsconfig": "^0.8.1", // Vue 官方 TypeScript 配置
4   "typescript": "~5.8.3", // TypeScript 编译器
5   "vite": "^7.1.7", // 现代构建工具
6   "vue-tsc": "^3.0.7" // Vue TypeScript 编译器
7 }
```

4. 项目结构分析

4.1. 目录结构

```
1 my-vue-app/
2   ├── .vscode/
3   │   └── extensions.json          # VS Code 推荐扩展
4   ├── public/
5   │   └── vite.svg                # 静态资源
6   ├── src/
7   │   ├── api/
8   │   │   └── auth.ts            # 认证相关API
9   │   ├── router/
10  │   │   └── index.ts           # 路由配置
11  │   ├── types/
12  │   │   └── api.ts             # TypeScript 类型定义
13  │   ├── utils/
14  │   │   └── request.ts         # HTTP 请求工具
```

```

15 |   |   └── views/
16 |   |       ├── Home.vue      # 首页组件
17 |   |       └── Login.vue     # 登录页组件
18 |   ├── App.vue              # 根组件
19 |   ├── main.ts               # 应用入口文件
20 |   └── style.css              # 全局样式
21 ├── docs/
22 |   └── project-setup-guide.md # 项目创建指南
23 ├── index.html                # HTML 模板
24 ├── package.json              # 项目配置和依赖
25 ├── tsconfig.json              # TypeScript 根配置
26 ├── tsconfig.app.json          # 应用 TypeScript 配置
27 ├── tsconfig.node.json         # Node.js TypeScript 配置
28 ├── vite.config.ts             # Vite 构建配置
29 └── README.md                  # 项目说明文档

```

4.2. 关键文件说明

4.2.1. package.json

- 作用: 项目元数据和依赖管理
- 重要字段:
 - `"type": "module"`: 启用 ES 模块支持
 - `scripts`: 定义项目脚本命令

4.2.2. vite.config.ts

```

1 import { defineConfig } from "vite";
2 import vue from "@vitejs/plugin-vue";
3
4 export default defineConfig({
5   plugins: [vue()],
6 });

```

- 作用: Vite 构建工具配置
- 为什么使用 Vite:
 - 极快的冷启动
 - 即时热模块替换 (HMR)
 - 基于 ESBUILD 的超快构建

4.2.3. TypeScript 配置文件

tsconfig.json (根配置)

- 继承 `@vue/tsconfig/tsconfig.dom.json`
- 引用应用和节点配置

tsconfig.app.json (应用配置)

- 配置 `src/` 目录下的 TypeScript 编译

- 包含 DOM 类型定义

tsconfig.node.json (Node.js 配置)

- 配置构建工具相关文件的 TypeScript 编译

5. 添加 UI 组件库

5.1. 安装 Element Plus 和图标库

```
1 npm install element-plus @element-plus/icons-vue
```

5.2. 为什么选择 Element Plus

1. **Vue 3 原生支持**: 专为 Vue 3 设计
2. **TypeScript 友好**: 完整的类型定义
3. **组件丰富**: 提供完整的企业级 UI 组件
4. **文档完善**: 中英文文档齐全
5. **社区活跃**: 持续维护和更新

5.3. 配置 Element Plus

在 `main.ts` 中全局引入:

```
1 import { createApp } from "vue";
2 import ElementPlus from "element-plus";
3 import "element-plus/dist/index.css";
4 import App from "./App.vue";
5
6 const app = createApp(App);
7 app.use(ElementPlus);
8 app.mount("#app");
```

6. 类型定义 (src/types/api.ts)

```

1  // 登录请求参数
2  export interface LoginRequest {
3      phone: string;
4      code: string;
5  }
6
7  // 统一响应数据类型
8  export interface ApiResponse {
9      code: number;
10     msg: string;
11     data: any;
12 }

```

为什么需要类型定义?

1. **类型安全**: 编译时检查, 减少运行时错误
2. **开发体验**: IDE 提供智能提示和自动补全
3. **代码可读性**: 清晰的数据结构定义
4. **重构友好**: 类型变更时自动检测相关代码

7. 配置 HTTP 请求

7.1. 创建 Axios 封装 (src/utils/request.ts)

```

1  /**
2   * HTTP 请求工具模块
3   * 基于 axios 封装, 提供统一的请求/响应处理、错误处理、认证等功能
4   */
5  import axios, {
6      type AxiosInstance,
7      type InternalAxiosRequestConfig,
8      type AxiosResponse,
9  } from "axios";
10 import { ElMessage } from "element-plus";
11 import type { ApiResponse } from "../types/api";
12
13 /**
14  * 创建 axios 实例
15  * 统一配置请求的基础设置, 包括 baseURL、超时时间、请求头等
16  */
17 const request: AxiosInstance = axios.create({
18     baseURL: "http://localhost:8888/api", // API 服务器地址
19     timeout: 10000, // 请求超时时间 10 秒
20     headers: {
21         "Content-Type": "application/json;charset=UTF-8", // 默认请求头, 指定内容类型为 JSON
22     },
23 });
24

```

```

25  /**
26   * 请求拦截器
27   * 在每次请求发送前执行，主要用于：
28   * 1. 添加认证 token 到请求头
29   * 2. 统一处理请求配置
30   * 3. 记录请求错误
31   */
32  request.interceptors.request.use(
33    (config: InternalAxiosRequestConfig) => {
34      // 从 localStorage 获取 token
35      const token = localStorage.getItem("token");
36
37      // 如果存在 token，则添加到请求头的 Authorization 字段
38      if (token && config.headers) {
39        config.headers.Authorization = `Bearer ${token}`;
40      }
41
42      return config;
43    },
44    (error) => {
45      // 请求发送前出错时的处理
46      console.error("请求错误:", error);
47      return Promise.reject(error);
48    }
49  );
50
51  /**
52   * 响应拦截器
53   * 在收到响应后执行，主要用于：
54   * 1. 统一处理响应数据
55   * 2. 根据业务状态码处理不同情况
56   * 3. 统一错误处理和用户提示
57   */
58  request.interceptors.response.use(
59    (response: AxiosResponse<ApiResponse>) => {
60      const { data } = response;
61
62      // 如果 HTTP 状态码为 200，说明网络请求成功
63      if (response.status === 200) {
64        return response;
65      }
66
67      // 其他状态码视为失败，显示错误信息并抛出异常
68      ElMessage.error(data.msg || "请求失败");
69      return Promise.reject(new Error(data.msg || "请求失败"));
70    },
71    (error) => {
72      // 处理 HTTP 错误状态码和网络错误
73      let message = "请求失败";
74
75      if (error.response) {
76        // 服务器返回了错误状态码

```

```

77     switch (error.response.status) {
78       case 401:
79         message = "未授权, 请重新登录";
80         // 401 表示 token 无效或过期, 清除本地存储的认证信息并跳转到登录页
81         localStorage.removeItem("token");
82         localStorage.removeItem("userPhone");
83         localStorage.removeItem("loginTime");
84         window.location.href = "/login";
85         break;
86       case 403:
87         message = "拒绝访问"; // 权限不足
88         break;
89       case 404:
90         message = "请求资源不存在"; // 接口不存在
91         break;
92       case 500:
93         message = "服务器内部错误"; // 服务器错误
94         break;
95       case 502:
96         message = "网关错误"; // 网关或代理服务器错误
97         break;
98       case 503:
99         message = "服务不可用"; // 服务暂时不可用
100        break;
101       case 504:
102         message = "网关超时"; // 网关超时
103         break;
104       default:
105         message = `请求失败: ${error.response.status}`;
106     }
107   } else if (error.request) {
108     // 请求已发送但没有收到响应 (网络错误)
109     message = "网络连接失败";
110   }
111   // 如果既没有 response 也没有 request, 则是请求配置出错, 使用默认错误信息
112
113   console.error("响应错误:", error);
114   ElMessage.error(message); // 向用户显示友好的错误提示
115   return Promise.reject(error);
116 }
117 );
118
119 // 导出配置好的 axios 实例, 供其他模块使用
120 export default request;

```

为什么这样封装?

1. **统一配置**: 集中管理 baseUrl、超时等配置
2. **拦截器**: 统一处理请求/响应, 添加 token、错误处理等
3. **类型安全**: TypeScript 提供完整类型检查
4. **可维护性**: 后续修改 HTTP 配置只需在一处修改

7.2. 创建 API 接口层 (src/api/auth.ts)

```

1  import request from "../utils/request";
2  import type { LoginRequest, ApiResponse } from "../types/api";
3
4  // 相关API - 只包含后端实际提供的接口
5  export const authApi = {
6    // 发送短信验证码
7    sendSms: (phone: string): Promise<ApiResponse> => {
8      return request.get(`/sms/send?phone=${phone}`).then((res) => res.data);
9    },
10
11    // 用户登录 (短信验证码登录)
12    login: (data: LoginRequest): Promise<ApiResponse> => {
13      return request.post("/login", data).then((res) => res.data);
14    },
15  };

```

7.3. 接口说明

1. 发送短信验证码 (**sendSms**):
 - 接收手机号参数
 - 调用后端发送验证码到用户手机
2. 短信验证码登录 (**login**):
 - 接收手机号和验证码
 - 验证成功后返回用户信息和 token

7.4. 使用方法

在组件中直接导入需要的 API:

```

1  // 在 Vue 组件或其他模块中使用
2  import { authApi } from '@/api/auth';
3
4  // 发送验证码
5  await authApi.sendSms('13800138000');
6
7  // 登录
8  await authApi.login({ phone: '13800138000', code: '123456' });

```

8. 配置路由 (src/router/index.ts)

```

1  import { createRouter, createWebHistory } from "vue-router";
2  import type { RouteRecordRaw } from "vue-router";
3
4  const routes: Array<RouteRecordRaw> = [
5    {

```



```

6     path: "/",
7     redirect: "/home",
8   },
9   {
10    path: "/login",
11    name: "Login",
12    component: () => import("../views/Login.vue"),
13  },
14  {
15    path: "/home",
16    name: "Home",
17    component: () => import("../views/Home.vue"),
18    meta: { requiresAuth: true },
19  },
20 ];
21
22 const router = createRouter({
23   history: createWebHistory(),
24   routes,
25 });
26
27 // 路由守卫
28 router.beforeEach((to, _from, next) => {
29   const token = localStorage.getItem("token");
30
31   if (to.meta.requiresAuth && !token) {
32     // 需要认证但没有token, 跳转到登录页
33     next("/login");
34   } else if (to.path === "/login" && token) {
35     // 已登录用户访问登录页, 跳转到首页
36     next("/home");
37   } else {
38     next();
39   }
40 });
41
42 export default router;

```

8.1. 路由配置说明

1. 路由定义:

- / 重定向到 /home
- /login 登录页面
- /home 首页, 需要认证

2. 懒加载: 使用 `() => import()` 实现路由组件懒加载, 优化首屏加载速度

3. 路由守卫:

- 检查 token 实现权限控制
- 未登录用户访问需要认证的页面会跳转到登录页
- 已登录用户访问登录页会跳转到首页

4. **Meta 字段:** `requiresAuth` 标识需要认证的路由

8.2. 在 main.ts 中注册路由

```
1 import { createApp } from "vue";
2 import ElementPlus from "element-plus";
3 import "element-plus/dist/index.css";
4 import App from "./App.vue";
5 import router from "./router";
6
7 const app = createApp(App);
8 app.use(ElementPlus);
9 app.use(router);
10 app.mount("#app");
```

9. 创建页面组件

9.1. 登录页面 (src/views/Login.vue)

- 手机号输入
- 验证码发送和输入
- 登录功能

```
1 <template>
2   <div class="login-container">
3     <el-card class="login-card">
4       <template #header>
5         <div class="card-header">
6           <el-icon><Iphone /></el-icon>
7           <span class="title">短信登录</span>
8         </div>
9       </template>
10
11     <el-form
12       @submit.prevent="login"
13       :model="formData"
14       ref="loginFormRef"
15       :rules="rules"
16     >
17       <el-form-item prop="phone">
18         <el-input
19           v-model="phone"
20           placeholder="请输入手机号"
21           size="large"
22           :prefix-icon="Iphone"
23           maxlength="11"
24           show-word-limit
25         />
26       </el-form-item>
```

```

27
28     <el-form-item>
29       <el-button
30         type="primary"
31         @click="sendSms"
32         :disabled="isCountingDown"
33         :loading="isSendingSms"
34         size="large"
35         style="width: 100%"
36       >
37         <el-icon><Message /></el-icon>
38         {{ isCountingDown ? `${countdown}秒后重发` : "发送验证码" }}
39       </el-button>
40     </el-form-item>
41
42     <div v-show="isCodeSent">
43       <el-form-item prop="code">
44         <el-input
45           v-model="code"
46           placeholder="请输入验证码"
47           size="large"
48           :prefix-icon="Lock"
49           maxlength="4"
50           show-word-limit
51         />
52       </el-form-item>
53
54       <el-form-item>
55         <el-button
56           type="success"
57           @click="login"
58           :loading="isLoggingIn"
59           size="large"
60           style="width: 100%"
61         >
62           <el-icon><Check /></el-icon>
63           登录
64         </el-button>
65       </el-form-item>
66     </div>
67
68     <el-alert
69       v-if="errorMessage"
70       :title="errorMessage"
71       type="error"
72       :closable="false"
73       show-icon
74     />
75
76     <el-alert
77       v-if="successMessage"
78       :title="successMessage"

```

```

79         type="success"
80         :closable="false"
81         show-icon
82     />
83     </el-form>
84 </el-card>
85 </div>
86 </template>
87
88 <script setup lang="ts">
89 import { ref, watch, reactive } from "vue";
90 import { useRouter } from "vue-router";
91 import { ElMessage, type FormInstance, type FormRules } from "element-plus";
92 import { Iphone, Message, Lock, Check } from "@element-plus/icons-vue";
93 import type { LoginRequest } from "../types/api";
94 import { authApi } from "../api/auth";
95
96 const router = useRouter();
97
98 const phone = ref("13951905171");
99 const code = ref("");
100 const errorMessage = ref("");
101 const successMessage = ref("");
102 const isCountingDown = ref(false);
103 const countdown = ref(60);
104 const isSendingSms = ref(false);
105 const isLoggingIn = ref(false);
106 let timer: ReturnType<typeof setInterval>;
107 const isCodeSent = ref(false);
108 const loginFormRef = ref<FormInstance>();
109
110 const formData = reactive({
111     phone,
112     code,
113 });
114
115 const rules: FormRules = {
116     phone: [
117         { required: true, message: "请输入手机号", trigger: "blur" },
118         {
119             pattern: /^1[3-9]\d{9}$/,
120             message: "请输入正确的手机号",
121             trigger: "blur",
122         },
123     ],
124     code: [
125         { required: true, message: "请输入验证码", trigger: "blur" },
126         { len: 4, message: "验证码长度为4位", trigger: "blur" },
127     ],
128 };
129
130 const sendSms = async () => {

```

```

131     if (!loginFormRef.value) return;
132
133     const valid = await loginFormRef.value
134       .validateField("phone")
135       .catch(() => false);
136     if (!valid) return;
137
138     if (isCountingDown.value) {
139       return;
140     }
141
142     errorMessage.value = "";
143     successMessage.value = "";
144     isSendingSms.value = true;
145
146     try {
147       const response = await authApi.sendSms(phone.value);
148
149       if (response.code === 0) {
150         successMessage.value = "验证码已发送!";
151         ElMessage.success("验证码已发送, 请查收!");
152         isCodeSent.value = true;
153         startCountdown();
154       } else {
155         errorMessage.value = response.msg || "发送失败, 请重试.";
156         ElMessage.error(response.msg || "发送失败, 请重试.");
157       }
158     } catch (error) {
159       const errorMsg = "请求失败, 请稍后再试.";
160       errorMessage.value = errorMsg;
161       ElMessage.error(errorMsg);
162     } finally {
163       isSendingSms.value = false;
164     }
165   };
166
167   const login = async () => {
168     if (!loginFormRef.value) return;
169
170     const valid = await loginFormRef.value.validate().catch(() => false);
171     if (!valid) return;
172
173     errorMessage.value = "";
174     successMessage.value = "";
175     isLoggingIn.value = true;
176
177     try {
178       const loginData: LoginRequest = {
179         phone: phone.value,
180         code: code.value,
181       };
182

```

```

183     const response = await authApi.login(loginData);
184     const loginResult = response.data;
185     console.log(loginResult);
186
187     if (loginResult.token) {
188         localStorage.setItem("token", loginResult.token);
189         localStorage.setItem("userPhone", loginResult.phone);
190         localStorage.setItem("loginTime", new Date().toISOString());
191
192         successMessage.value = loginResult.message || "登录成功!";
193         ElMessage.success(loginResult.message || "登录成功! ");
194
195         // 跳转到首页
196         router.push("/home");
197     } else {
198         errorMessage.value = loginResult.message || "登录失败, 请重试.";
199         ElMessage.error(loginResult.message || "登录失败, 请重试.");
200     }
201 } catch (error) {
202     const errorMsg = "请求失败, 请稍后再试.";
203     errorMessage.value = errorMsg;
204     ElMessage.error(errorMsg);
205 } finally {
206     isLoggingIn.value = false;
207 }
208 };
209
210 const startCountdown = () => {
211     isCountingDown.value = true;
212     countdown.value = 60; // 重置倒计时为60秒
213
214     timer = setInterval(() => {
215         if (countdown.value > 0) {
216             countdown.value--;
217         } else {
218             clearInterval(timer);
219             isCountingDown.value = false;
220         }
221     }, 1000);
222 };
223
224 // 清除定时器以避免内存泄漏
225 watch(isCountingDown, (newValue) => {
226     if (!newValue) {
227         clearInterval(timer);
228     }
229 });
230 </script>
231
232 <style scoped>
233 .login-container {
234     min-height: 100vh;

```

```
235     display: flex;
236     align-items: center;
237     justify-content: center;
238     background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
239     padding: 20px;
240 }
241
242 .login-card {
243     width: 100%;
244     max-width: 400px;
245     box-shadow: 0 15px 35px rgba(0, 0, 0, 0.1);
246     border-radius: 15px;
247     overflow: hidden;
248 }
249
250 .card-header {
251     display: flex;
252     align-items: center;
253     justify-content: center;
254     gap: 8px;
255     font-size: 20px;
256     font-weight: 600;
257     color: #409eff;
258 }
259
260 .card-header .el-icon {
261     font-size: 24px;
262 }
263
264 .title {
265     background: linear-gradient(45deg, #409eff, #67c23a);
266     -webkit-background-clip: text;
267     -webkit-text-fill-color: transparent;
268     background-clip: text;
269 }
270
271 :deep(.el-form-item) {
272     margin-bottom: 20px;
273 }
274
275 :deep(.el-input__wrapper) {
276     border-radius: 8px;
277 }
278
279 :deep(.el-button) {
280     border-radius: 8px;
281     font-weight: 500;
282     transition: all 0.3s ease;
283 }
284
285 :deep(.el-button:hover) {
286     transform: translateY(-1px);
```

```

287     box-shadow: 0 6px 20px rgba(0, 0, 0, 0.15);
288   }
289
290   :deep(.el-alert) {
291     border-radius: 8px;
292     margin-top: 10px;
293   }
294
295   :deep(.el-card__header) {
296     background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
297     border-bottom: 1px solid #e4e7ed;
298   }
299
300   :deep(.el-card__body) {
301     padding: 30px;
302   }
303 </style>

```

9.2. 首页 (src/views/Home.vue)

- 用户信息显示
- 退出登录功能

```

1   <template>
2     <div class="home-container">
3       <el-container>
4         <el-header class="header">
5           <div class="header-content">
6             <h2 class="title">欢迎来到首页</h2>
7             <div class="user-info">
8               <el-button type="primary" @click="logout">
9                 <el-icon><SwitchButton /></el-icon>
10                退出登录
11              </el-button>
12            </div>
13          </div>
14        </el-header>
15
16        <el-main class="main-content">
17          <el-card class="welcome-card">
18            <template #header>
19              <div class="card-header">
20                <el-icon><User /></el-icon>
21                <span>用户信息</span>
22              </div>
23            </template>
24
25            <div class="user-profile">
26              <el-descriptions :column="1" size="large">
27                <el-descriptions-item label="手机号">
28                  {{ userPhone }}

```



```

29         </el-descriptions-item>
30         <el-descriptions-item label="登录时间">
31             {{ loginTime }}
32         </el-descriptions-item>
33         <el-descriptions-item label="Token状态">
34             <el-tag type="success">已认证</el-tag>
35         </el-descriptions-item>
36     </el-descriptions>
37 </div>
38 </el-card>
39
40 <el-card class="feature-card">
41     <template #header>
42         <div class="card-header">
43             <el-icon><Setting /></el-icon>
44             <span>功能区域</span>
45         </div>
46     </template>
47
48     <el-empty description="功能开发中..." />
49 </el-card>
50 </el-main>
51 </el-container>
52 </div>
53 </template>
54
55 <script setup lang="ts">
56 import { ref, onMounted } from "vue";
57 import { useRouter } from "vue-router";
58 import { ElMessage } from "element-plus";
59 import { SwitchButton, User, Setting } from "@element-plus/icons-vue";
60
61 const router = useRouter();
62 const userPhone = ref("");
63 const loginTime = ref("");
64
65 onMounted(() => {
66     // 获取存储的用户信息
67     const phone = localStorage.getItem("userPhone");
68     const time = localStorage.getItem("loginTime");
69
70     if (phone) {
71         userPhone.value = phone;
72     }
73
74     if (time) {
75         loginTime.value = new Date(time).toLocaleString();
76     } else {
77         loginTime.value = new Date().toLocaleString();
78     }
79 });
80

```

```
81  const logout = () => {
82    // 清除本地存储
83    localStorage.removeItem("token");
84    localStorage.removeItem("userPhone");
85    localStorage.removeItem("loginTime");
86
87    ElMessage.success("退出登录成功");
88
89    // 跳转到登录页
90    router.push("/login");
91  };
92  </script>
93
94  <style scoped>
95    .home-container {
96      width: 100%;
97      height: 100vh;
98      background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
99    }
100
101    .header {
102      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
103      box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
104    }
105
106    .header-content {
107      display: flex;
108      justify-content: space-between;
109      align-items: center;
110      height: 100%;
111      max-width: 1200px;
112      margin: 0 auto;
113      padding: 0 20px;
114    }
115
116    .title {
117      color: white;
118      margin: 0;
119      font-size: 24px;
120      font-weight: 600;
121    }
122
123    .user-info {
124      display: flex;
125      align-items: center;
126    }
127
128    .main-content {
129      width: 100%;
130      padding: 20px;
131    }
132
```

```

133 .welcome-card,
134 .feature-card {
135     margin-bottom: 20px;
136     border-radius: 12px;
137     box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
138     width: 100%;
139 }
140
141 .card-header {
142     display: flex;
143     align-items: center;
144     gap: 8px;
145     font-size: 18px;
146     font-weight: 600;
147     color: #409eff;
148 }
149
150 .card-header .el-icon {
151     font-size: 20px;
152 }
153
154 .user-profile {
155     padding: 20px 0;
156 }
157
158 :deep(.el-descriptions__label) {
159     font-weight: 600;
160     color: #606266;
161 }
162
163 :deep(.el-descriptions__content) {
164     color: #303133;
165 }
166
167 :deep(.el-button) {
168     border-radius: 8px;
169     font-weight: 500;
170     transition: all 0.3s ease;
171 }
172
173 :deep(.el-button:hover) {
174     transform: translateY(-1px);
175     box-shadow: 0 6px 20px rgba(0, 0, 0, 0.15);
176 }
177
178 :deep(.el-card__header) {
179     background: linear-gradient(135deg, #f8f9fa 0%, #e9ecef 100%);
180     border-bottom: 1px solid #e4e7ed;
181 }
182 </style>

```

10. 项目脚本命令

package.json 中的 scripts

```
1  {
2    "scripts": {
3      "dev": "vite", // 开发服务器
4      "build": "vue-tsc -b && vite build", // 构建生产版本
5      "preview": "vite preview" // 预览构建结果
6    }
7  }
```

命令说明:

- `npm run dev` : 启动开发服务器, 支持热重载
- `npm run build` :
 - `vue-tsc -b` : TypeScript 类型检查
 - `vite build` : 构建优化的生产版本
- `npm run preview` : 本地预览构建后的应用

11. VS Code 扩展推荐

.vscode/extensions.json

```
1  {
2    "recommendations": [
3      "Vue.volar", // Vue 官方扩展
4      "Vue.vscode-typescript-vue-plugin" // TypeScript Vue 插件
5    ]
6  }
```

为什么推荐这些扩展

1. **Vue.volar**: 提供 Vue 3 完整支持, 包括模板语法高亮、智能提示等
2. **Vue.vscode-typescript-vue-plugin**: 增强 TypeScript 在 Vue 文件中的支持

12. 项目总结

12.1. 技术选型优势

1. **Vue 3**:
 - Composition API 提供更好的逻辑复用
 - 更好的 TypeScript 支持
 - 更小的包体积和更好的性能

2. Vite:

- 开发时极快的冷启动
- 基于 ESBUILD 的快速构建
- 原生 ES 模块支持

3. TypeScript:

- 静态类型检查
- 更好的开发体验
- 大型项目维护优势

4. Element Plus:

- 企业级 UI 组件
- 完善的设计规范
- 开箱即用

12.2. 后续开发建议

1. **状态管理**: 项目复杂时考虑添加 Pinia
2. **测试**: 添加 Vitest 进行单元测试
3. **代码规范**: 配置 ESLint 和 Prettier
4. **路由守卫**: 添加认证和权限控制
5. **错误边界**: 添加全局错误处理
6. **性能优化**: 代码分割、懒加载等